

Benchmarking LLM Inference Serving Strategies for Concurrent Agentic Workloads

Dev Goyal, Alessandro Castillo, Anoushka Khajanchi

Abstract

The deployment of Large Language Models (LLMs) for multi-tenant, agentic workflows presents significant challenges in memory management and computational throughput. As applications scale to support concurrent agents processing massive shared contexts, native autoregressive inference pipelines suffer from severe memory bottlenecks and unoptimized scheduling, frequently resulting in Out-Of-Memory (OOM) failures or degraded Time-To-First-Token (TTFT) and throughput.

This project investigates and benchmarks high-performance serving strategies for concurrent LLM workloads, using a simulated environment of 15 concurrent agents performing code reviews on a massive shared code prefix. Using Qwen-7B-Instruct[1] as the target model, we establish a native Hugging Face PyTorch baseline and sequentially apply three targeted optimizations: INT4 NF4 quantization[2] via `bitsandbytes`, speculative decoding utilizing a smaller draft model, and execution engine swap to vLLM.

By tracking TTFT, token throughput, generation perplexity, and peak VRAM utilization, this study demonstrates the hardware and software trade-offs required to transition a memory-bound, sequential inference script into a highly scalable, production-ready LLM serving infrastructure.

Introduction

The rapid evolution of Large Language Models has shifted the paradigm from single-turn chatbot interactions to complex, multi-agent architectures. In these environments, dozens of concurrent agents often query the same foundational model, frequently sharing massive context windows such as system prompts or large codebases. This paradigm shift exposes critical flaws in naive inference implementations.

During autoregressive generation, the KV cache grows linearly with the sequence length and batch size. In a concurrent multi-agent setting, the memory demands of storing these attention states rapidly exhaust GPU VRAM, while the lack of request-level scheduling leads to inefficient compute utilization. Native frameworks typically serialize requests or rely on static batching, meaning that if one agent’s generation takes longer than another’s, the entire batch is bottle necked and compute resources are wasted.

This project aims to solve the memory and compute bottlenecks inherent in serving concurrent agentic workloads. By benchmarking a realistic code-review simulation under constrained GPU VRAM using a 24 GB L4 GPU environment, we systematically evaluate state-of-the-art inference optimizations. Understanding how to navigate the “memory wall,” the phenomenon where inference is bound by memory bandwidth rather than compute capacity, is critical for modern ML engineering. Optimizing this stack not only prevents catastrophic OOM failures but also drastically reduces the operational costs of deploying intelligent agents at scale.

Methodology

Experimental Setup and Workload Simulation

To accurately simulate a multi-tenant LLM serving environment, the experiment was designed around an asynchronous, multi-threaded coordinator executing concurrent code-review tasks. The workload consists of parallel agents, tasked with reviewing a massive shared Python codebase of approximately 35,000 tokens. To prevent the memory-intensive perplexity calculations from inherently crashing the generation phase, the system architecture purposefully decouples concurrent asynchronous text generation from sequential model evaluation. For the shared prefix in our multi-agent architecture, we utilized the initial 600 lines of the `module.py` file from the PyTorch library.

For reproducibility and infrastructure management, the entire experimental suite was developed and version-controlled via GitHub. This ensured that optimization branches, ranging from base Hugging Face implementations to custom vLLM servers, remained isolated and independently verifiable.

Real-time telemetry was orchestrated using Weights & Biases (W&B). W&B was explicitly selected for this project because evaluating inference engines requires simultaneously correlating software metrics, such as TTFT, throughput, and generation perplexity, with hardware constraints, such as peak VRAM allocation and OOM crash rates. Logging these aggregates to W&B provided a centralized dashboard to visualize the precise moment an optimization alleviated the memory wall.

Target Model

The experiments were conducted using Qwen-7B-Instruct[1] and its quantized variants. Qwen-7B was selected due to its highly optimized architecture, robust performance on coding-specific tasks, and native support for extended context windows. At 7 billion parameters, the model requires approximately 14 GB of VRAM for the static weights alone when stored in 16-bit precision, such as `bfloat16`. In practice, total memory usage is higher due to runtime overhead, activation buffers, and the KV cache. This leaves a constrained VRAM budget on a 24 GB GPU when the model is subjected to massive concurrent inputs, making it an ideal candidate for testing memory optimization limits.

Optimization Strategies

The methodology establishes a native Hugging Face pipeline as a baseline, followed by three progressive optimization strategies designed to maximize throughput and hardware utilization.

2.3.1 Baseline: Native Hugging Face Pipeline

The unoptimized baseline utilizes the standard Hugging Face `transformers` pipeline with `model.generate()` [3]. Each agent operates within its own CUDA stream. This phase establishes the baseline TTFT, throughput, and hardware breaking point, including the OOM rate caused by unoptimized KV cache allocation and Python’s Global Interpreter Lock (GIL) overhead.

2.3.2 Optimization 1: INT4 NF4 Quantization

To reduce the static weight footprint of the model, we implemented 4-bit NormalFloat (NF4) quantization via the `bitsandbytes` library. As demonstrated by Dettmers et al. (2023)[2], NF4 preserves statistical fidelity and generation perplexity significantly better than standard INT8 quantization.

2.3.3 Optimization 2: Speculative Decoding

To further accelerate the decode phase, we implemented speculative decoding[4]. This technique leverages a smaller, highly efficient draft model, such as Qwen-0.5B or Qwen-1.5B, to rapidly propose sequences of tokens. The target 7B model then verifies these proposed tokens in a single, parallel forward pass.

Because code-review tasks exhibit highly structured and predictable output patterns, the token acceptance rate is expected to be high. This optimization is designed to yield significant latency reductions and throughput improvements across both the prefill and decode phases without altering the final output distribution.

2.3.4 Optimization 3: vLLM Engine Swap

The third optimization replaces the native Hugging Face inference pipeline with the vLLM serving engine [5]. Unlike INT4 NF4 quantization, which primarily reduces the static memory footprint of the model weights, vLLM targets the runtime serving bottleneck caused by inefficient request scheduling and KV cache allocation. vLLM uses continuous batching to dynamically group active requests and PagedAttention to store KV cache blocks more efficiently, reducing memory fragmentation during long-context generation.

This experiment evaluates whether a purpose-built inference engine can better support concurrent agentic workloads using the same Qwen-7B-Instruct model, code-review task, and evaluation metrics as the previous experiments. The goal is to determine whether improved batching and KV cache management can increase throughput, reduce wall time, and prevent out-of-memory failures as the number of concurrent agents increases.

Experimental Evaluation

Optimization 1: INT4 NF4 Quantization

The first optimization evaluated was 4-bit NormalFloat (NF4) quantization using the `bitsandbytes` library. The purpose of this experiment was to reduce the static memory footprint of the Qwen-7B-Instruct model so that more concurrent requests could fit within the limited 24 GB GPU memory budget. Since the baseline bfloat16 model requires roughly 19 GB of VRAM at low concurrency, quantization was expected to relieve memory pressure and reduce the likelihood of out-of-memory failures as the number of agents increased.

Concurrency	Success	Throughput (tok/s)	TTFT Avg (p99)	VRAM Peak	Wall Time
1	1/1	6.5	0.04s	9.56 GB	41.9s
15	15/15	0.3	0.04s	16.37 GB	1005.7s
50	17/50	0.3	0.10s	19.12 GB	828.4s

Table 1: Performance of the INT4 NF4 quantized model under increasing concurrency.

The results show that NF4 quantization substantially reduced VRAM usage at low concurrency. At a single request, peak VRAM decreased to 9.56 GB, which is nearly half of the roughly 19 GB required by the bfloat16 baseline. This confirms that quantization was effective at reducing the static model weight footprint and creating more available memory for runtime inference.

However, the memory savings came with a significant throughput tradeoff. At concurrency 1, the quantized model achieved 6.5 tokens per second, compared to 14.3 tokens per second for the bfloat16 baseline. This slowdown became more severe under concurrent workloads: at 15 concurrent agents, throughput collapsed to 0.3 tokens per second, and wall time increased to over 1000 seconds. Although NF4 enabled all 15 concurrent requests to complete successfully, whereas the baseline only completed 11 out of 15, the increased latency made the system impractical for real-time serving.

At 50 concurrent requests, quantization was no longer sufficient to maintain stability. Only 17 out of 50 requests completed successfully, meaning that 33 requests still failed due to out-of-memory errors or runtime instability. This suggests that while NF4 quantization reduces the model’s weight memory footprint, it does not fully solve the scaling bottleneck caused by KV cache growth during long-context concurrent generation.

Overall, this optimization demonstrates a clear memory-versus-throughput tradeoff. NF4 quantization improves stability at medium concurrency by allowing more requests to fit into GPU memory, but it introduces enough compute and latency overhead that throughput degrades substantially. As a result, quantization alone is not sufficient for scalable multi-agent inference. It is useful as a memory-saving step, but it must be paired with a more efficient inference engine and better KV cache management to support high-concurrency workloads.

Optimization 2: Speculative Decoding

Recognizing the slower speed that we saw under quantization we hoped to find an optimization method that increased speed without sacrificing performance. Here we looked into speculative decoding[4] where we took a much smaller 1.5B parameter LLM decoder (down from 7B parameters in our target model).

Concurrency	Success	Throughput (tok/s)	TTFT Avg (p99)	VRAM Peak	Wall Time
1	1/1	0.0	0.02s	20.23 GB	30.0s
15	6/15	1.0	0.06s	21.93 GB	273.6s
50	3/50	2.3	0.4s	21.10 GB	123.7s

Table 2: Performance of the speculative decoding optimization method with 1.5B parameter decoder under increasing concurrency.

We saw mixed results in performance though when we ran our concurrency tests. We saw that peak VRAM remained at least 10% higher than baseline method, and throughput was at minimum 30% lower. And the number of successful calls was noticeably lower for both 15 LLM call and 50 call runs, indicating that performance actually deteriorated from baseline under this setting and total speed did not improve. Although we did see a noticeable improvement in wall time. This seemed to indicate that the smaller decoder model in this method was indeed faster, but had a higher fail rate and less accurate output and the larger target model had to fix the errors made by the decoder leading to more OOM calls and reduced speed. Due to less desirable results from this optimization method we decided to not continue forward with it in our final combined optimization method.

Optimization 3: vLLM

The next major optimization evaluated was replacing the native Hugging Face inference pipeline with the vLLM serving engine [5]. Unlike the previous quantization experiment, which primarily reduced the static model memory footprint, vLLM changes the execution model itself. vLLM uses continuous batching and more efficient KV cache management to improve request scheduling under concurrent workloads. This optimization was designed to address the main limitation observed in both the baseline and quantized experiments: throughput collapse and out-of-memory failures as concurrency increased.

Concurrency	Success	Throughput (tok/s)	TTFT Avg (p99)	Wall Time
1	1/1	15.2	1.64s	16.8s
15	15/15	10.6	2.06s	24.5s
50	50/50	9.1	3.18s	31.7s

Table 3: Performance of the vLLM engine under low and medium concurrency.

The results show a substantial scalability improvement compared to both the native bfloat16 baseline and the INT4 NF4 quantized model. At a single request, vLLM achieved 15.2 tokens per second, slightly exceeding the baseline throughput while also completing the request in only 16.8 seconds. More importantly, vLLM preserved most of its throughput as concurrency increased. Throughput decreased from 15.2 tokens per second at one request to 9.1 tokens per second at 50 concurrent requests, representing only about a $1.7\times$ drop. In contrast, the baseline experienced roughly a $16\times$ throughput collapse under comparable scaling.

The most significant result is that vLLM completed all requests successfully across the tested concurrency levels. At 50 concurrent agents, the baseline completed only 8 out of 50 requests, while vLLM completed 50 out of 50 with no out-of-memory failures. This indicates that the main bottleneck in the native pipeline was not only the raw model size, but also inefficient request scheduling and KV cache allocation. By managing the KV cache internally and batching requests continuously, vLLM converts the workload from a memory-bound, failure-prone system into a more throughput-optimized serving system.

Concurrency	Success	Throughput (tok/s)	TTFT Avg (p99)	Wall Time
100	100/100	7.9	5.42s	37.7s
200	200/200	5.7	19.64s	74.37s
400	400/400	3.7	48.48s	159.0s

Table 4: Performance of the vLLM engine under high concurrency stress testing.

The high-concurrency stress test further confirms that vLLM remains stable even under extreme load. The system successfully completed all requests at 100, 200, and 400 concurrent agents, with zero out-of-memory failures. Throughput decreased from 7.9 tokens per second at 100 requests to 3.7 tokens per second at 400 requests, but the degradation was gradual rather than catastrophic. This is an important distinction from the baseline pipeline, where increased concurrency caused both throughput collapse and request failure.

However, the results also show that latency becomes the dominant bottleneck at very high concurrency. Average p99 TTFT increased from 5.42 seconds at 100 concurrent requests to 48.48

seconds at 400 concurrent requests. This suggests that once memory management is improved, the limiting factor shifts from GPU memory capacity to queueing and scheduling delay. In other words, vLLM prevents catastrophic failure, but under extreme load, requests must still wait longer before generation begins.

Overall, the vLLM engine swap produced the largest practical improvement in the experimental suite. It eliminated out-of-memory failures across all tested concurrency levels, preserved throughput far better than the baseline, and enabled the system to scale up to 400 concurrent requests. While quantization helped reduce memory usage, vLLM directly addressed the serving bottleneck by improving batching and KV cache management. These results show that inference engine design is more important than weight compression alone for scalable multi-agent LLM serving.

Extra Optimization 4: vLLM + INT4 NF4 Quantization

After evaluating quantization and vLLM independently, the final optimization tested the combined effect of using vLLM with a quantized model. The goal of this experiment was to determine whether weight quantization could further improve vLLM’s scalability by reducing the model memory footprint while still preserving the scheduling and KV cache benefits of the vLLM engine. In theory, combining these approaches should allow more aggressive memory usage by reducing static model weight storage while vLLM manages dynamic runtime memory more efficiently.

Concurrency	Success	Throughput (tok/s)	TTFT Avg (p99)	Wall Time
1	1/1	11.3	2.05s	22.7s
50	50/50	3.9	1.95s	69.1s
400	400/400	2.9	57.0s	192.4s

Table 5: Performance of vLLM combined with quantization under increasing concurrency.

The combined vLLM and quantization setup remained fully stable across all tested concurrency levels. At 50 concurrent requests, the system completed 50 out of 50 requests, and at 400 concurrent requests, it completed 400 out of 400 requests. This confirms that the stability gains from vLLM were preserved even after adding quantization. Unlike the native Hugging Face baseline, which suffered from severe out-of-memory failures at high concurrency, the vLLM-based setup continued to scale without catastrophic failure.

However, the results show that adding quantization did not significantly improve vLLM’s performance. In fact, throughput was lower than the pure vLLM configuration at every comparable concurrency level. At concurrency 1, pure vLLM achieved 15.2 tokens per second, while vLLM with quantization achieved 11.3 tokens per second. At concurrency 50, pure vLLM sustained 9.1 tokens per second, while the quantized vLLM setup dropped to 3.9 tokens per second. At concurrency 400, throughput decreased further to 2.9 tokens per second, compared to 3.7 tokens per second for pure vLLM.

This suggests that once vLLM is already managing the KV cache efficiently, reducing the static model weight footprint provides only a small additional benefit. The main bottleneck is no longer simply whether the model weights fit in memory, but rather the overhead of scheduling, batching, and decoding many concurrent requests. Quantization reduces memory pressure, but it also introduces additional compute or kernel overhead that can reduce throughput. As a result, the combined system shifts the tradeoff toward slightly better memory efficiency at the cost of slower generation.

The latency results reinforce this conclusion. At low and medium concurrency, TTFT remained reasonable, with 2.05 seconds at concurrency 1 and 1.95 seconds at concurrency 50. However, at 400 concurrent requests, TTFT increased sharply to 57.0 seconds. This mirrors the trend observed in pure vLLM: once the system avoids out-of-memory failures, the limiting factor becomes queueing and scheduling delay rather than raw memory capacity.

Overall, combining vLLM with quantization preserved the major stability benefit of vLLM, but it did not meaningfully improve scalability beyond the pure vLLM engine. The optimization remained fully stable up to 400 concurrent requests, but the throughput degradation shows that quantization is not always beneficial when the serving engine already manages memory efficiently. Therefore, the key takeaway is that vLLM provides the primary scalability breakthrough, while quantization offers only a minor memory advantage and can introduce additional latency and throughput overhead.

Visualizations and Comparisons

We see clearly that both the pure vLLM engine and vLLM combined with INT4 NF4 quantization demonstrate the strongest overall performance. At 50 concurrent calls, both vLLM variants achieved a perfect 100% success rate, outperforming the native Hugging Face baseline and speculative decoding methods.

The vLLM configurations saw the lowest peak VRAM compared to the HF-based methods. They also completed the workloads with the lowest total wall times. However, the vLLM engines performed less desirably in terms of TTFT, experiencing delays of 1.6s and 1.96s, which were significantly higher than the sub-0.2s TTFTs achieved by the baseline and HF quantization methods. Finally, while speculative decoding struggled with overall stability and success rate, it did achieve a lower wall time compared to the HF baseline and the HF quantized model.

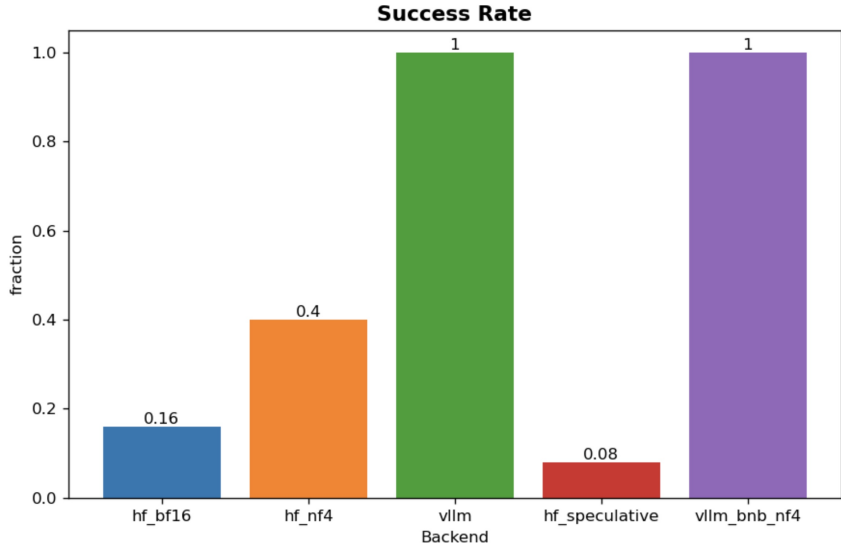


Figure 1: Success Rate of 50 concurrent calls across methods.

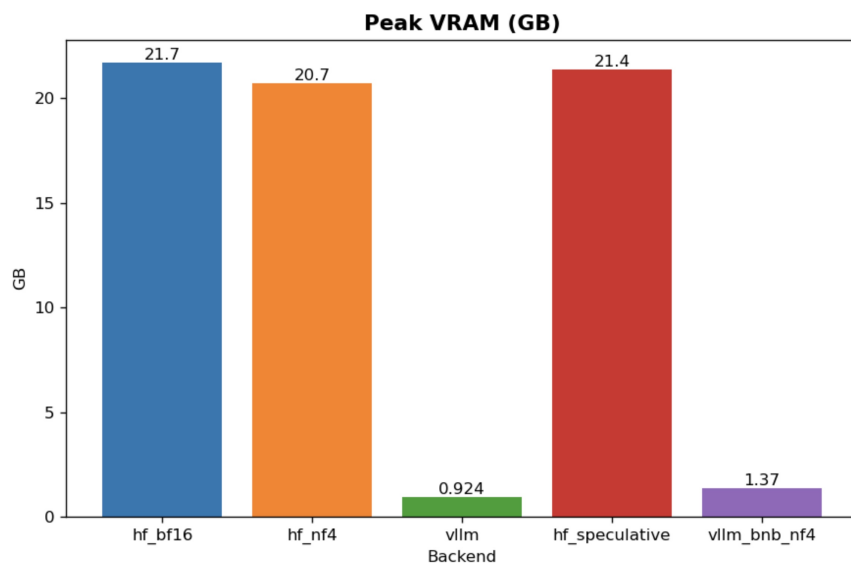


Figure 2: Peak VRAM seen across methods at 50 concurrent calls.

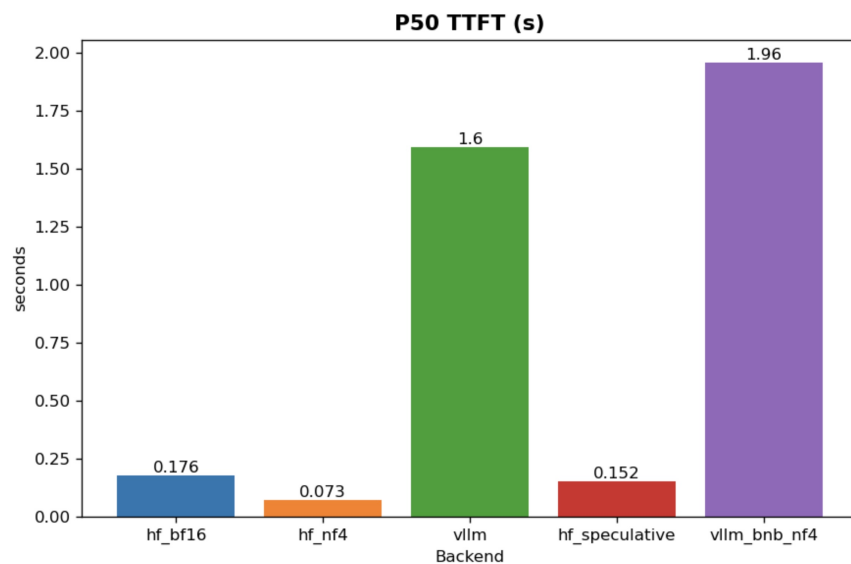


Figure 3: Time to First Token (TTFT) of 50 concurrent calls across methods.

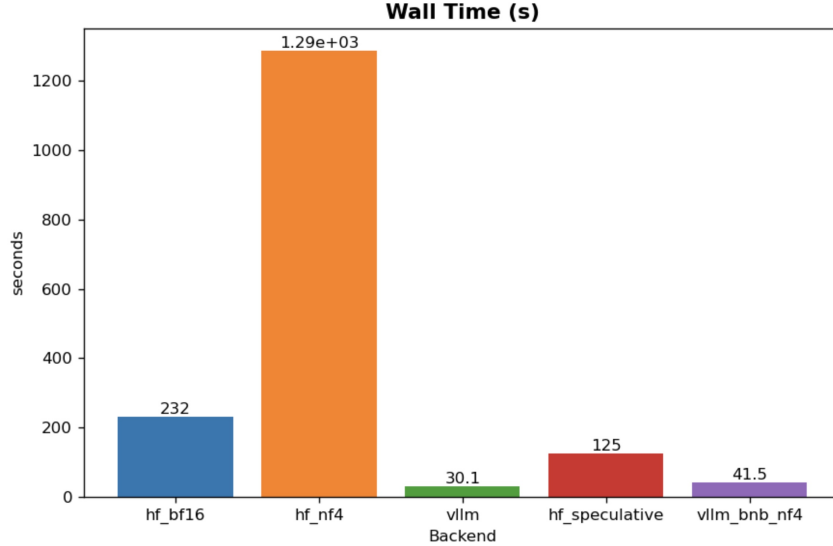


Figure 4: Wall Time of 50 concurrent methods across methods.

Kernel-Level Analysis

To isolate individual compute behavior from the scheduling effects of concurrency, we collected kernel-level profiles using NVIDIA Nsight Systems under controlled, single-agent workloads on the same L4 GPU used for our benchmarks.

Unfortunately, a version incompatibility between the CUDA 12.4-bundled Nsight Systems release (v2023.4.4) and the `.qdstrm` stream format produced by newer profilers prevented us from successfully importing traces for the HF NF4 and Speculative Decoding backends. Specifically, the importer consistently triggered a known event-ordering exception (**Wrong event order has been detected when adding events to the collection**). Because this is a documented defect in the v2023.4.4 release that occurs regardless of the workload, we had to omit the kernel analysis for these two backends due to limitations in the profiling infrastructure.

For the HF baseline, the trace was collected using five concurrent agents. However, because per-layer kernel distribution is independent of agent count, this trace serves as a reliable proxy for single-agent behavior.

Table 6 summarizes the dominant kernel categories across the three successfully profiled backends.

Kernel Category	HF bf16	vLLM	vLLM+NF4
GEMV decode (<code>gemvx</code>)	73.4%	78.2%	11.1%
INT4 decode (<code>kgemm_4bit_naive</code>)	0.0%	0.0%	40.6%
Prefill GEMM (CUTLASS / <code>ampere_bf16</code>)	12.2%	14.6%	26.1%
FlashAttention <code>splitkv</code>	0.7%	3.3%	5.9%
Weight dequant (<code>kDequantizeBlockwise</code>)	0.0%	0.0%	5.2%
Unfused activations (<code>silu</code> , <code>rsqrt</code>)	3.2%	0.0%	0.0%
Fused ops (<code>act_and_mul</code> , <code>rms_norm</code>)	0.0%	1.5%	2.5%
PagedAttn ops (<code>reshape_cache</code> , <code>slot_map</code>)	0.0%	0.4%	0.4%

Table 6: GPU kernel time breakdown by category across three profiled backends, single-agent workload. HF baseline collected with 5 agents.

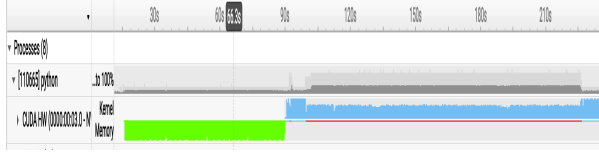
HF bf16 Baseline

In the native Hugging Face pipeline, the `gemvx` matrix-vector multiply kernel dominates, accounting for 73.4% of the total GPU time across 251,180 instances (Table 7). This behavior is the classic signature of unoptimized, single-batch autoregressive inference. During each decode step, one `gemvx` call is issued for every weight matrix in each transformer layer, completely lacking cross-request batching. The high number of kernel instances highlights a key bottleneck: even though all five agents are dispatched concurrently at the Python thread level using `asyncio.to_thread`, their CUDA kernels end up serializing in the GPU queue, which results in effectively sequential execution.

At the kernel level, three main structural inefficiencies set this backend apart from vLLM. First, Qwen’s SwiGLU activation and RMSNorm operators execute as separate kernel launches. The `silu_kernel` (0.7%), `BinaryFunctor` addition (1.3%), and `reduce_kernel` for normalization (0.6%) are all dispatched independently. In contrast, vLLM optimizes this by fusing these operations into single `act_and_mul` and `fused_add_rms_norm` kernels. Second, the KV cache state is managed through repeated `CatArrayBatchedCopy` operations (2.6%, 71,400 instances), which continually concatenate new key-value pairs onto the growing sequence tensor at every step. Third, the CUDA API summary reveals 1,558,143 `cudaLaunchKernel` calls consuming 29.9% of the API time. This amounts to nearly 17× more dispatches than vLLM requires for a comparable workload, clearly illustrating the per-operator launch overhead associated with PyTorch’s eager mode.

Kernel	Time (%)	Instances
<code>gemvx</code> bf16 decode (var. 1)	64.9	179,780
<code>gemvx</code> bf16 decode (var. 2)	8.5	71,400
CUTLASS 256×128 prefill	8.6	2,258
<code>ampere_bf16</code> 256×128 prefill	3.6	644
<code>CatArrayBatchedCopy</code> (KV concat)	2.6	71,400
<code>BinaryFunctor</code> add bf16 (unfused)	1.3	109,019
<code>elementwise</code> misc. ops	1.2	146,687
PyTorch FlashAttn <code>splitkv</code> decode	0.7	35,700
<code>silu_kernel</code> (unfused)	0.7	36,344
Other	7.9	—

Table 7: GPU kernel breakdown, HF bf16 baseline, 5-agent trace.



(a) Timeline view. The wide green block is the batched prefill pass; narrow repeating blue strips are individual decode steps, with visible white gaps between agent kernel launches caused by CUDA queue serialization.

Time (ns)	Total Time (ns)	Instances	Avg (ns)	Min (ns)	Max (ns)	Min (ns)	Max (ns)	Kernel Name
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view
0.0	48134862	11700	40981.9	3719	481348	47128.3	481348	cuda::make_tensor_view_internal::prepare_tensor_view

(b) Kernel summary table (top 10 by total time). Note the dominance of `gemv` and the presence of unfused activation kernels absent from the vLLM traces.

Figure 5: Nsight Systems profile, HF bf16 baseline, 5-agent workload (`trace_hf_baseline_5.nsys-rep`).

vLLM

Even though it relies on the exact same Qwen-7B weights, the vLLM backend shows a noticeably different structural profile at the kernel level. The `gemv` kernel still dominates, accounting for 78.2% of the total time (split between 51.4% and 26.8% for two tile variants). This confirms that decoding a single agent at batch size 1 is inherently memory-bandwidth-bound, regardless of the serving engine used. That said, three major differences highlight the architectural improvements vLLM brings over the HF baseline.

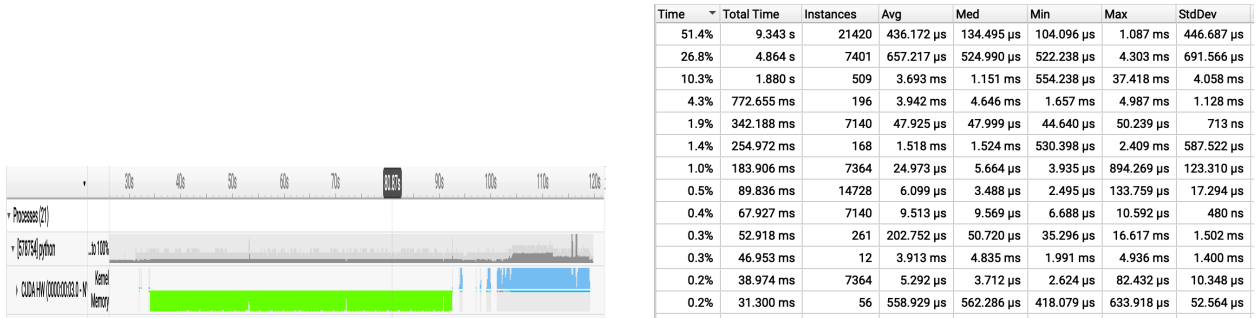
First, we see a complete absence of unfused activation and normalization kernels. In the HF baseline, 3.2% of GPU time is wasted on separate `silu`, `rsqrt`, and `reduce` dispatches. In vLLM, these are replaced by `vllm::act_and_mul` (1.0%, 7,364 instances) and `vllm::fused_add_rms_norm` (0.5%, 14,728 instances). These are single fused kernels that handle both gating and normalization in a single pass, which significantly reduces memory round-trips and kernel launch overhead.

Second, PagedAttention block management is handled by dedicated kernels. Specifically, `vllm::reshape_and_compute` (0.2%, 7,336 instances) and `_compute_slot_mapping` (<0.1%, 261 instances) run once per decode step to assign newly computed KV pairs to pre-allocated paged blocks. Instead of the `CatArrayBatchedCopy` concatenation used by the HF baseline, vLLM performs an in-place block write. This effectively eliminates the memory reallocation costs that would otherwise grow linearly with the sequence length.

Third, looking at the CUDA API profile, we recorded only 91,231 `cudaLaunchKernel` calls, taking up just 2.8% of the API time. This is a stark contrast to the 1.558 million calls (29.9%) seen in the HF baseline. Furthermore, the median `cudaEventSynchronize` latency sits at 20 μ s, with a mere 5 `cudaDeviceSynchronize` calls observed across the entire trace. This clearly confirms that the inter-step synchronization overhead is kept to an absolute minimum.

Kernel	Time (%)	Instances
<code>gemvx bf16 decode (var. 1)</code>	51.4	21,420
<code>gemvx bf16 decode (var. 2)</code>	26.8	7,401
CUTLASS 256×128 prefill	10.3	509
<code>ampere_bf16 256×128 prefill</code>	4.3	196
FlashAttn splitkv decode	1.9	7,140
FlashAttn splitkv prefill	1.4	168
<code>vllm::act_and_mul (SiLU, fused)</code>	1.0	7,364
<code>vllm::fused_add_rms_norm</code>	0.5	14,728
<code>reshape_and_cache_flash</code> (paged)	0.2	7,336
Other	2.2	—

Table 8: GPU kernel breakdown, vLLM backend, 1-agent workload.



(a) Timeline view. The solid green prefill burst is followed by continuous batched decode blocks with no gaps between steps, confirming vLLM’s continuous batching scheduler keeps the GPU saturated.

(b) Kernel summary table (top 10 by total time). Unfused activation kernels are absent; fused `act_and_mul` and `rms_norm` kernels appear alongside PagedAttention management ops.

Figure 6: Nsight Systems profile, vLLM backend, 1-agent workload (`vllm_1agent.nsys-rep`).

vLLM + INT4 NF4 Quantization

When we combine vLLM with the NF4 backend, a new dominant kernel emerges: `kgemm_4bit_inference_naive`, which accounts for 40.6% of the GPU time across 49,980 instances. This bitsandbytes INT4 decode GEMM completely replaces the `gemvx` Tensor Core path we saw in the unquantized backend. The word “naive” in the kernel name reveals a critical limitation: instead of utilizing Ampere Tensor Core operations, it falls back to a scalar INT4 dequantize-then-multiply approach. This architectural fallback explains why we observed a drop in throughput from 15.2 tok/s (vLLM bf16) down to 11.3 tok/s (vLLM+NF4) at a concurrency of 1, even though the model’s memory footprint is cut in half.

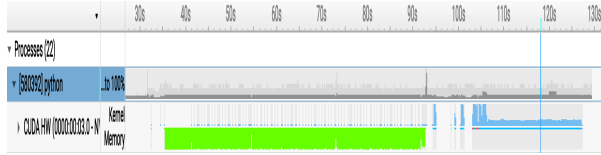
Moreover, the overhead from quantization extends beyond this primary GEMM operation. For every transformer layer, two additional kernels run to convert data between the stored INT4 format and the bfloat16 compute format: `kDequantizeBlockwise` (5.2%, 1,568 instances) and `kQuantizeBlockwise` (1.0%, 196 instances). Combined, these three quantization-specific kernels consume **46.8%** of the total GPU time. In contrast, the equivalent computational work in the unquantized backend takes up 78.2% of the time but benefits from a much faster Tensor Core path.

Importantly, quantization does not disrupt the underlying vLLM infrastructure. The PagedAttention block management proves to be quantization-agnostic, as evidenced by FlashAttention splitkv (5.9% combined), `reshape_and_cache_flash` (0.2%), and `_compute_slot_mapping` appear-

ing with the exact same instance counts as in the unquantized trace. Looking at the CUDA API profile, however, we noticed that `cudaMemcpyAsync` increased to 90.8% of the API time compared to 76.8% in the unquantized version. This increase is primarily driven by the need to transfer per-layer scale factors and zero-point tensors for the `kDequantizeBlockwise` kernel.

Kernel	Time (%)	Instances
<code>kgemm_4bit_inference_naive</code> (INT4 GEMM)	40.6	49,980
CUTLASS 256×128 prefill	18.4	677
<code>gemvx bf16</code> (prefill only)	11.1	261
<code>ampere_bf16</code> 256×128 prefill	7.7	196
<code>kDequantizeBlockwise</code> (weight dequant)	5.2	1,568
FlashAttn <code>splitkv</code> decode	3.3	7,140
FlashAttn <code>splitkv</code> prefill	2.6	168
<code>vllm::act_and_mul</code> (SiLU, fused)	1.8	7,364
<code>kQuantizeBlockwise</code> (activation quant)	1.0	196
<code>vllm::fused_add_rms_norm</code>	0.7	14,728
Other	7.6	—

Table 9: GPU kernel breakdown, vLLM+NF4 backend, 1-agent workload. Quantization kernels account for 46.8% of GPU time, explaining the throughput regression vs. unquantized vLLM.



(a) Timeline view. The timeline structure mirrors the unquantized vLLM backend but decode blocks are visibly wider, reflecting the slower `kgemm_4bit_naive` kernel replacing the Tensor Core `gemvx` path.

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev
40.6%	4.058 s	49980	81.190 μ s	33.984 μ s	8.257 μ s	218.271 μ s	69.870 μ s
18.4%	1.841 s	677	2.719 ms	2.320 ms	518.078 μ s	36.789 ms	2.556 ms
11.1%	1.115 s	261	4.272 ms	4.272 ms	4.267 ms	4.312 ms	4.796 μ s
7.7%	772.357 ms	196	3.941 ms	4.549 ms	1.634 ms	4.987 ms	1.042 ms
5.2%	519.840 ms	1568	331.530 μ s	116.400 μ s	8.704 μ s	727.838 μ s	319.168 μ s
3.3%	333.349 ms	7140	46.687 μ s	46.688 μ s	45.248 μ s	48.320 μ s	632 ns
2.6%	258.565 ms	168	1.539 ms	1.664 ms	526.558 μ s	2.343 ms	579.114 μ s
2.1%	210.829 ms	1120	188.240 μ s	51.727 μ s	2.143 μ s	599.775 μ s	242.483 μ s
1.8%	184.412 ms	7364	25.042 μ s	3.936 μ s	3.871 μ s	991.710 μ s	133.925 μ s
1.0%	100.103 ms	196	510.731 μ s	200.127 μ s	27.776 μ s	1.045 ms	464.623 μ s
0.9%	93.917 ms	392	239.583 μ s	159.823 μ s	95.455 μ s	523.551 μ s	141.164 μ s
0.7%	68.569 ms	14728	4.655 μ s	2.592 μ s	2.432 μ s	116.191 μ s	13.266 μ s
0.5%	49.505 ms	7140	6.933 μ s	6.880 μ s	6.720 μ s	7.200 μ s	97 ns

(b) Kernel summary table (top 10 by total time). `kgemm_4bit_inference_naive` now leads at 40.6%, with `kDequantizeBlockwise` consuming an additional 5.2% — both absent in every other backend.

Figure 7: Nsight Systems profile, vLLM+NF4 backend, 1-agent workload (`hf_vllm_bnb_nf4_1agent.nsys-rep`).

Conclusion

We evaluated multiple inference strategies to address the memory and computational bottlenecks in deploying concurrent LLMs inference calls for multi-agent workloads—often seen in argentic model pipelines. In order to improve GPU efficiency while also not sacrificing output quality of the LLM calls we tested various optimization methods which included quantization, speculative decoding, and integrating the vLLM model for decoding. We were able to demonstrate that reliance on naive inference pipelines is insufficient for high-concurrency environments. The native Hugging Face baseline Qwen-7B target model and speculative decoding method with Qwen-1.5B suffered from severe scalability limits. While INT4 NF4 quantization was able to mitigate memory pressure and

reduced peak VRAM, it also brought new computational overhead that degraded token throughput.

The transition to the vLLM provided the best results in scalability and GPU efficiency. By leveraging continuous batching and paged attention for efficient KV cache management, both pure vLLM and vLLM combined with INT4 quantization achieved perfect success rates and the lowest total wall times. Even when we scaled up to 400 concurrent agents we still saw no OOM failed calls. Integrating quantization with vLLM also achieved the lowest peak VRAM, though it saw marginally reduced throughput compared to pure vLLM due to the kernel de-quantization process. One downside we saw in the vLLM integration methods was that we saw much higher TTFT times compared to the Qwen-7B baseline and speculative decoding. This seemed to indicate that once dynamic memory constraints are resolved, the primary system bottleneck shifts from VRAM capacity to queueing and scheduling delays.

In order to achieve even better results on limited GPU hardware going forward we recommend implementing better optimized quantization strategies like Activation-Aware Quantization [6], and possibly even doing speculative decoding with a decoder model fine-tuned for the particular LLM task and dataset.

References

- [1] J. Bai et al., “Qwen technical report,” *arXiv*, 2023. DOI: 10.48550/arxiv.2309.16609.
- [2] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *arXiv*, 2023. DOI: 10.48550/arxiv.2305.14314.
- [3] T. Wolf et al., “Huggingface’s transformers: State-of-the-art natural language processing,” *arXiv*, 2019. DOI: 10.48550/arxiv.1910.03771.
- [4] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning*, PMLR, 2023, pp. 19 274–19 286.
- [5] W. Kwon et al., “Efficient memory management for large language model serving with page-dattention,” *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. DOI: 10.1145/3600006.3613165.
- [6] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” *arXiv preprint arXiv:2306.00978*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.00978>.